

31.5-2

Find all integers x that leave remainders 1, 2, and 3 when divided by 9, 8, and 7, respectively.

31.5-3

Argue that, under the definitions of Theorem 31.27, if $\gcd(a, n) = 1$, then

$$(a^{-1} \bmod n) \leftrightarrow ((a_1^{-1} \bmod n_1), (a_2^{-1} \bmod n_2), \dots, (a_k^{-1} \bmod n_k)) .$$

31.5-4

Under the definitions of Theorem 31.27, prove that for any polynomial f , the number of roots of the equation $f(x) = 0 \pmod{n}$ equals the product of the number of roots of each of the equations $f(x) = 0 \pmod{n_1}$, $f(x) = 0 \pmod{n_2}$, ..., $f(x) = 0 \pmod{n_k}$.

31.6 Powers of an element

Along with considering the multiples of a given element a , modulo n , we often consider the sequence of powers of a , modulo n , where $a \in \mathbb{Z}_n^*$:

$$a^0, a^1, a^2, a^3, \dots,$$

modulo n . Indexing from 0, the 0th value in this sequence is $a^0 \bmod n = 1$, and the i th value is $a^i \bmod n$. For example, the powers of 3 modulo 7 are

i	0	1	2	3	4	5	6	7	8	9	10	11	...
$3^i \bmod 7$	1	3	2	6	4	5	1	3	2	6	4	5	...

and the powers of 2 modulo 7 are

i	0	1	2	3	4	5	6	7	8	9	10	11	...
$2^i \bmod 7$	1	2	4	1	2	4	1	2	4	1	2	4	...

In this section, let $\langle a \rangle$ denote the subgroup of $\mathbb{Z}_n^*$ generated by a through repeated multiplication, and let $\text{ord}_n(a)$ (the “order of a , modulo n ”) denote the order of a in $\mathbb{Z}_n^*$. For example, $\langle 2 \rangle = \{1, 2, 4\}$ in

$\mathbb{Z}_7^*$, and $\text{ord}_7(2) = 3$. Using the definition of the Euler phi function $\phi(n)$ as the size of $\mathbb{Z}_n^*$ (see Section 31.3), we now translate Corollary 31.19 into the notation of $\mathbb{Z}_n^*$ to obtain Euler's theorem and specialize it to $\mathbb{Z}_p^*$, where p is prime, to obtain Fermat's theorem.

Theorem 31.30 (Euler's theorem)

For any integer $n > 1$,

$$a^{\phi(n)} = 1 \pmod{n} \text{ for all } a \in \mathbb{Z}_n^*.$$

■

Theorem 31.31 (Fermat's theorem)

If p is prime, then

$$a^{p-1} = 1 \pmod{p} \text{ for all } a \in \mathbb{Z}_p^*.$$

Proof By equation (31.22), $\phi(p) = p - 1$ if p is prime.

■

Fermat's theorem applies to every element in $\mathbb{Z}_p$ except 0, since $0 \notin \mathbb{Z}_p^*$. For all $a \in \mathbb{Z}_p$, however, we have $a^p = a \pmod{p}$ if p is prime.

If $\text{ord}_n(g) = |\mathbb{Z}_n^*|$, then every element in $\mathbb{Z}_n^*$ is a power of g , modulo n , and g is a **primitive root** or a **generator** of $\mathbb{Z}_n^*$. For example, 3 is a primitive root, modulo 7, but 2 is not a primitive root, modulo 7. If $\mathbb{Z}_n^*$ possesses a primitive root, the group $\mathbb{Z}_n^*$ is **cyclic**. We omit the proof of the following theorem, which is proven by Niven and Zuckerman [345].

Theorem 31.32

The values of $n > 1$ for which $\mathbb{Z}_n^*$ is cyclic are 2, 4, p^e , and $2p^e$, for all primes $p > 2$ and all positive integers e .

■

If g is a primitive root of $\mathbb{Z}_n^*$ and a is any element of $\mathbb{Z}_n^*$, then there exists a z such that $g^z = a \pmod{n}$. This z is a **discrete logarithm** or an **index** of a , modulo n , to the base g . We denote this value as $\text{ind}_{n,g}(a)$.

Theorem 31.33 (Discrete logarithm theorem)

If g is a primitive root of $\mathbb{Z}_n^*$, then the equation $g^x = g^y \pmod{n}$ holds if and only if the equation $x = y \pmod{\phi(n)}$ holds.

Proof Suppose first that $x = y \pmod{\phi(n)}$. Then, we have $x = y + k\phi(n)$ for some integer k , and thus

$$\begin{aligned} g^x &= g^{y+k\phi(n)} \pmod{n} \\ &= g^y \cdot (g^{\phi(n)})^k \pmod{n} \\ &= g^y \cdot 1^k \pmod{n} \quad (\text{by Euler's theorem}) \\ &= g^y \pmod{n}. \end{aligned}$$

Conversely, suppose that $g^x = g^y \pmod{n}$. Because the sequence of powers of g generates every element of $\langle g \rangle$ and $|\langle g \rangle| = \phi(n)$, Corollary 31.18 implies that the sequence of powers of g is periodic with period $\phi(n)$. Therefore, if $g^x = g^y \pmod{n}$, we must have $x = y \pmod{\phi(n)}$. ■

Let's now turn our attention to the square roots of 1, modulo a prime power. The following properties will be useful to justify the primality-testing algorithm in Section 31.8.

Theorem 31.34

If p is an odd prime and $e \geq 1$, then the equation

$$x^2 = 1 \pmod{p^e} \tag{31.33}$$

has only two solutions, namely $x = 1$ and $x = -1$.

Proof By Exercise 31.6-2, equation (31.33) is equivalent to

$$p^e \mid (x - 1)(x + 1).$$

Since $p > 2$, we can have $p \mid (x - 1)$ or $p \mid (x + 1)$, but not both. (Otherwise, by property (31.3), p would also divide their difference $(x + 1) - (x - 1) = 2$.) If $p \nmid (x - 1)$, then $\gcd(p^e, x - 1) = 1$, and by Corollary

31.5, we would have $p^e \mid (x + 1)$. That is, $x \equiv -1 \pmod{p^e}$. Symmetrically, if $p \nmid (x + 1)$, then $\gcd(p^e, x + 1) = 1$, and Corollary 31.5 implies that $p^e \mid (x - 1)$, so that $x \equiv 1 \pmod{p^e}$. Therefore, either $x \equiv -1 \pmod{p^e}$ or $x \equiv 1 \pmod{p^e}$. ■

A number x is a *nontrivial square root of 1, modulo n* , if it satisfies the equation $x^2 \equiv 1 \pmod{n}$ but x is equivalent to neither of the two “trivial” square roots: 1 or -1 , modulo n . For example, 6 is a nontrivial square root of 1, modulo 35. We’ll use the following corollary to Theorem 31.34 in Section 31.8 to prove the Miller-Rabin primality-testing procedure correct.

Corollary 31.35

If there exists a nontrivial square root of 1, modulo n , then n is composite.

Proof By the contrapositive of Theorem 31.34, if there exists a nontrivial square root of 1, modulo n , then n cannot be an odd prime or a power of an odd prime. Nor can n be 2, because if $x^2 \equiv 1 \pmod{2}$, then $x \equiv 1 \pmod{2}$, and therefore, all square roots of 1, modulo 2, are trivial. Thus, n cannot be prime. Finally, we must have $n > 1$ for a nontrivial square root of 1 to exist. Therefore, n must be composite. ■

Raising to powers with repeated squaring

A frequently occurring operation in number-theoretic computations is raising one number to a power modulo another number, also known as *modular exponentiation*. More precisely, we would like an efficient way to compute $a^b \pmod{n}$, where a and b are nonnegative integers and n is a positive integer. Modular exponentiation is an essential operation in many primality-testing routines and in the RSA public-key cryptosystem. The method of *repeated squaring* solves this problem efficiently.

Repeated squaring is based on the following formula to compute a^b for nonnegative integers a and b :

$$a^b = \begin{cases} 1 & \text{if } b = 0, \\ (a^{b/2})^2 & \text{if } b > 0 \text{ and } b \text{ is even}, \\ a \cdot a^{b-1} & \text{if } b > 0 \text{ and } b \text{ is odd}. \end{cases} \quad (31.34)$$

The last case, where b is odd, reduces to the one of the first two cases, since if b is odd, then $b - 1$ is even. The recursive procedure MODULAR-EXPONENTIATION on the next page computes $a^b \bmod n$ using equation (31.34), but performing all computations modulo n . The term “repeated squaring” comes from squaring the intermediate result $d = a^{b/2}$ in line 5. Figure 31.4 shows the values of the parameter b , the local variable d , and the value returned at each level of the recursion for the call MODULAR-EXPONENTIATION(7,560,561), which returns the result 1.

b	560	280	140	70	35	34	17	16	8	4	2	1	0
d	67	166	298	241	355	160	103	526	157	49	7	1	–
returned value	1	67	166	298	241	355	160	103	526	157	49	7	1

Figure 31.4 The values of the parameter b , the local variable d , and the value returned for recursive calls of MODULAR-EXPONENTIATION with parameter values $a = 7$, $b = 560$, and $n = 561$. The value returned by each recursive call is assigned directly to d . The result of the call with $a = 7$, $b = 560$, and $n = 561$ is 1.

```

MODULAR-EXPONENTIATION( $a, b, n$ )
1 if  $b == 0$ 
2   return 1
3 elseif  $b \bmod 2 == 0$ 
4    $d = \text{MODULAR-EXPONENTIATION}(a, b/2, n)$  //  $b$  is even
5   return  $(d \cdot d) \bmod n$ 
6 else  $d = \text{MODULAR-EXPONENTIATION}(a, b - 1, n)$  //  $b$  is odd
7   return  $(a \cdot d) \bmod n$ 

```

The total number of recursive calls depends on the number of bits of b and the values of these bits. Assume that $b > 0$ and that the most significant bit of b is a 1. Each 0 generates one recursive call (in line 4), and each 1 generates two recursive calls (one in line 6 followed by one in line 4 because if b is odd, then $b - 1$ is even). If the inputs a , b , and n are β -bit numbers, then there are between β and $2\beta - 1$ recursive calls altogether, the total number of arithmetic operations required is $O(\beta)$, and the total number of bit operations required is $O(\beta^3)$.

Exercises

31.6-1

Draw a table showing the order of every element in $\mathbb{Z}_{11}^*$. Pick the smallest primitive root g and compute a table giving $\text{ind}_{11,g}(x)$ for all $x \in \mathbb{Z}_{11}^*$.

31.6-2

Show that $x^2 = 1 \pmod{p^e}$ is equivalent to $p^e \mid (x - 1)(x + 1)$.

31.6-3

Rewrite the third case of MODULAR-EXPONENTIATION, where b is odd, so that if b has β bits and the most significant bit is 1, then there are always exactly β recursive calls.

31.6-4

Give a nonrecursive (i.e., iterative) version of MODULAR-EXPONENTIATION.

31.6-5

Assuming that you know $\phi(n)$, explain how to compute $a^{-1} \pmod{n}$ for any $a \in \mathbb{Z}_n^*$ using the procedure MODULAR-EXPONENTIATION.

31.7 The RSA public-key cryptosystem